

Название задачи:

Онлайн-магазин «Мобильный мир» - Проектирование схем коллекций для шардирования данных

Специфика MongoDB:

- коллекция = таблица;
- документ = строка;
- на уровне бд реализована возможность шардинга по хешу, по диапазону;
- CP: Consistency - Partition Tolerance
- бд предоставляет набор атомарных операций чтение-апдейт;
- конкурентное обновление: запросы выстраиваются в очередь.

Функциональные требования

№	Действующие лица или системы	Use Case	Описание
UC1	Пользователь, микросервис Заказов	Заказ пользователя из Москвы может включать товары из категорий «Электроника» и «Книги».	Коллекция orders включает заказы клиентов и содержит атрибуты: - Уникальный идентификатор заказа; - Идентификатор клиента; - Дату и время оформления заказа; - Список заказанных товаров и их цену; - Статус заказа; - Общую сумму заказа; - Геозону заказа. <i>Основные операции:</i> - Быстрое создание заказов с одновременным списанием остатков. - Поиск истории заказов конкретного пользователя. - Отображение статуса заказа.
UC2	Пользователь, микросервис Продуктов	Система должна отслеживать остатки продуктов по гео-зонам. Например, в Екатеринбурге есть в наличии 50 штук товара «Смартфон X», а в Калининграде — 30.	Коллекция products хранит сведения о товарах и включает такие атрибуты: - Уникальный идентификатор товара; - Наименование; - Категорию товара; - Цену; - Остаток товара в каждой геоzone; - Дополнительные атрибуты (цвет, размер). <i>Основные операции:</i> - Частые обновления остатков при покупках. - Поиск товаров по категориям и фильтрация по диапазону цен. - Описание товара на странице продукта.
UC3	Пользователь, микросервис Корзина	Пользователю, как залогиненному, так и нет, доступны операции с корзиной товаров.	Коллекция carts хранит данные о текущих корзинах и включает атрибуты: - Уникальный идентификатор корзины; - Идентификатор пользователя и session_id для гостей; - Список товаров; - Статус корзины; - Дату и время создания; - Дату и время последнего обновления; - Время удаления. <i>Основные операции:</i> - Создание корзины, когда заходит гость или новый пользователь. - Получение текущей корзины по фильтру { session_id, status:"active" } или { user_id, status:"active" }. - Добавление или замена товара в корзине. - Удаление товара из корзины. - Слияние гостевой корзины в пользовательскую, если пользователь

№	Действующие лица или системы	Use Case	Описание
			залогинится: (1. прочитать гостевую { session_id, status:"active" }; 2. добавить её items в корзину { user_id, status:"active" }; 3. отметить гостевую как abandoned.). - Отметка корзины как заказанной.

Нефункциональные требования

№	Требование
1.	Система должна быть устойчивой к нагрузке во время распродаж.
2.	Модернизация системы должна пройти "бесшовно"; пользователи должны сохранить все возможности, предоставляемые существующей версией системы.
3.	Решение должно расширять существующее решение на Mongo DB.
4.	Модернизацию предполагается прооводить путем масштабирования БД.

Решение

Описание существующей коллекции Products

E Product
product_id: number «generated»
- name: text - category: text - price: number - geo_leftovers: array --[{"zone":"Moscow", "qty": 10}] - attributes: json --{"color":"red", "size": 5}

Профиль нагрузки:

- коллекция содержит ограниченное количество документов;
- очень частые чтения по категории, цене;
- сам каталог товаров обновляется значительно реже, чем читается;
- массив остатков продукта обновляется очень часто, "конкурентными" процессами.

Решение	Техническая реализация	Обоснование
1. Отделить динамическую	Вертикальное партиционирование по	В существующей коллекции Products совмещены статические и динамические, часто изменяющиеся,

Решение	Техническая реализация	Обоснование
информацию об остатках от статической информации о продукте.	geo_leftovers	данные продуктов. Это выглядит неэффективным с точки зрения возможности кэширования не часто изменяющейся информации о продукте. В текущем виде коллекцию сложно эффективно шардировать.

Итак, предлагается разделить коллекцию на две, каждая из которых может быть эффективно масштабирована горизонтально:

 Product
• product_id: number «generated»
• name: text • category: text • price: number - attributes: json

Коллекция Products

- содержит редко изменяющиеся данные, следовательно, их можно эффективно кэшировать (Redis/CDN/...);
- при большом объеме продукции к коллекции Products можно применить шардирование по категории товара (чтобы сохранить эффективный поиск по категории):

```
sh.shardCollection(
  "db.Products",
  {category: "hashed"}
)
```

 ProductLeftover
• product_id: number «generated»
• geo: text • quantity: number • updated_at: timestamp

Коллекция ProductLeftovers

- содержит часто изменяющиеся данные;
- может быть шардирована по product_id - что даст равномерное распределение данных.
- может быть шардирована по гео-зоне - что даст изоляцию нагрузки по регионам:

```
sh.shardCollection(
  "db.ProductLeftovers",
```

```
{geo: 1, product_id: 1}  
)
```

Пример атомарного списания остатков:

```
db.ProductLeftovers.findOneAndUpdate(  
  { product_id: X, geo: "Moscow", quantity: { $gte: qty } },  
  { $inc: { quantity: -qty } }  
)
```

Недостатки, ограничения, риски

1. Разделение коллекций потребует дополнительные правки кода, а также миграцию исторических данных, что может привести к увеличению сроков модернизации, а также к потере/порче критических данных об остатках.
2. Шардирование коллекции Products по категории товара может привести к неравномерному распределению данных, если кол-во товаров в категориях сильно отличается.
3. Шардирование коллекции ProductLeftovers по гео-зоне может привести к неравномерному распределению данных, если одна гео-зона сильно отличается по кол-ву покупателей/интенсивности покупок.

Описание существующей коллекции Orders

 Order
• order_id: number «generated»
• user_id: number • created_at: timestamp • products: array --[{"product_id": 123, "qty": 1, "price": 500}] • status: text • total_sum: number • geo: text

Профиль нагрузки:

- коллекция содержит исторические данные и актуальные заказы авторизованного пользователя;
- заказы создаются постоянно => объем данных постоянно растет;
- исторические заказы, преимущественно, читаются (история по пользователю)

Решение	Техническая реализация	Обоснование
1. Обеспечить локальность заказов одного пользователя.	Шардирование по user_id + created_at.	Хешированное шардирование даст равномерное распределение данных. Все заказы одного пользователя окажутся на одном шарде => поиск истории пользователя будет происходить локально.
2. Как альтернатива - обеспечить локальность заказов внутри гео-зоны.	Шардирование по geo + created_at.	Тут нужно смотреть на бизнес-логику процессинга заказов.

Пример шардинга по пользователю:

```
sh.shardCollection(  
  "db.Orders",  
  {user_id: "hashed", created_at: 1}  
)  
...  
find({user_id: X}).sort({created_at: -1})
```

Пример шардинга по гео:

```
sh.shardCollection(  
  "db.Orders",  
  {geo: 1, created_at: 1}  
)
```

Описание существующей коллекции Carts

E Cart
• cart_id: number «generated»
user_id: number session_id: number • items: array --[{"product_id": 123, "qty": 1}] • status: text • created_at: timestamp • expires_at: timestamp updated_at: timestamp

Профиль нагрузки:

- корзина относится либо к авторизованному пользователю, либо к посетителю;
- возможна миграция корзины от посетителя к авторизованному пользователю;
- одна корзина содержит ограниченное кол-во товаров;
- корзины создаются постоянно, но так же постоянно трансформируются в заказы/удаляются по ttl => объем таблицы не растет на долгой дистанции;
- частое обновление статуса и списка товаров.

Решение	Техническая реализация	Обоснование	Недостатки
1. Ничего не менять.		Если логика подразумевает хард-делит старых корзин по ttl, то на долгой дистанции объем коллекции не растет. Соответственно для поиска достаточно индексирования данных.	На распродажах может быть всплеск создания и операций над корзинами. Надо анализировать реальную нагрузку. Тут нужно смотреть на бизнес-логику процессинга заказов.

Решение	Техническая реализация	Обоснование	Недостатки
2. Как альтернатива - распределение по пользователю.	Шардирование по user_id + status.	Если мы примем решение шардировать заказы по user_id, то неплохо бы шардировать и корзины по тому же пользователю. Это обеспечит локальность при формировании заказа из корзины.	Корзина может принадлежать неавторизованному посетителю => такие корзины будут располагаться на одной шард-ноде, что опять же грозит всплеском нагрузки при распродажах.
3. Как альтернатива - распределение по суррогатному идентификатору.	Шардирование по owner_id + status.	Корзина может относиться как к авторизованному пользователю, так и к неавторизованному посетителю.	Усложняем систему; дополнительная разработка.
4. Как альтернатива - добавить в коллекцию данные о гео.	Шардирование по geo + status.	Гео-зону авторизованного пользователя мы знаем. У посетителя мы можем попросить предварительно выбрать зону доставки, либо определить по ip.	Возможно неравномерное распределение данных.

Название задачи:

Онлайн-магазин «Мобильный мир» - Выявление и устранение «горячих» шардов

Функциональные требования

№	Действующие лица или системы	Use Case	Описание
UC1	Система мониторинга	Из-за категории «Электроника» произошла перегрузка одного из шардов MongoDB.	Система мониторинга должна обнаруживать "горячие" шарды.
UC2	Инструменты SRE	Товары из популярных категорий могут создавать непропорциональную нагрузку на отдельные узлы	Инженеры по обеспечению доступности должны иметь механизмы автоматического перераспределения данных.

Нефункциональные требования

№	Требование
1.	Предполагается решение на базе Mongo DB.

Решение

Проблему раздутых шардов по причине **категории** товаров мы минимизировали на первом этапе, отделив информацию об остатках от основной коллекции продуктов:

 Product	 ProductLeftover
• product_id: number «generated»	• product_id: number «generated»
• name: text • category: text • price: number • attributes: json	• geo: text • quantity: number • updated_at: timestamp

Коллекция Products может быть эффективно кеширована, а коллекция ProductLeftovers - шардирована по product_id:

```
sh.shardCollection(  
  "db.ProductLeftovers",  
  {product_id: "hashed"}  
)
```

Но в целом проблема горячих шардов может возникнуть по другим причинам, например, из-за популярного товара.

Для детектирования перегруженной шард-ноды можно предложить

1. Метрики нагрузки на шард: CPU, DiskIO, WiredTiger Cache Pressure, оценить состояние очереди конкурентных апдейтов. Для оценки использовать как встроенные инструменты mongo Например:

```
db.serverStatus().metrics.cpu  
db.serverStatus().wiredTiger.cache["percentage of cache used"]
```

так и настроить экспорт в Prometheus.

2. Метрики дисбаланса данных: число и размеры чанков на шарде, Chunk Migration Rate. Например:

```
sh.status()  
sh.printShardingStatus()
```

3. Метрики распределения ключей шардирования: частота использования ключа, распределение ключей. С помощью:

```
db.ProductLeftovers.aggregate()
```

4. Метрики производительности операций: 95, 99-перцентили операций обновления. Например:

```
db.serverStatus().opLatencies
```

5. Метрики репликации: оценить лаг репликации. Например:

```
rs.printSlaveReplicationInfo()
```

Механизмы автоматического перераспределения данных

1. Встроенный балансировщик MongoDB

```
sh.setBalancerState(true)
sh.setBalancerWindow("01:00", "05:00") // мигрировать ночью
```

или запускать балансировщик при нагрузке.

2. Auto-Split

```
sh.enableAutoSplit()
```

либо разрезание чанка по заданным правилам.

- 3. Range Tagging - привязать определенные товары к конкретным шард-нодам.
- 4. Использование Atlas-инсталляции и соответствующих инструментов балансировки данных.
- 5. Автоматическая оркестрация - ввод новых шардов при возрастании нагрузки+перевесбалансировка.

Название задачи:

Онлайн-магазин «Мобильный мир» - Настройка чтения с реплик и консистентность

Нефункциональные требования

№	Требование
1.	Пользователи не должны иметь проблемы из-за лага репликации
2.	Предполагается решение на базе Mongo DB.

Решение

Будем рассматривать реализованное ранее решение - коллекция ProductLeftovers отделена от Products.

Операция чтения	Тип реплики	Допустимый лаг репликации	Обоснование
Основные характеристики товара	secondary	до 30-60 сек	Эти данные почти не меняются, небольшая задержка не влияет на продажу.
Поиск товаров по категории/фильтрам	secondary	до 30-60 сек	Эти данные почти не меняются, небольшая задержка не влияет на продажу.
Цена товара в карточке	secondary	до 10 сек	Цены могут обновляться, но не критично быстро. Небольшая задержка допустима.
Актуальная цена в момент оформления заказа	primary		Лаг репликации недопустим, тк нужна актуальная цена.
Информация об остатках\гео-доступности в карточке товара	secondary	до 2-5 сек	Небольшой лаг не повлияет на продажи.
Проверка остатка при добавлении товара в корзину	primary		Если бизнес-логика подразумевает резервирование товара, то чтение должно быть максимально актуальным.
Проверка остатка при оформлении заказа	primary		Критична актуальность данных. Операция проверки и списания остатков должна быть атомарной.
История заказов пользователя	secondary	до 5–10 сек	Допустима легкая задержка.
Детали заказа	secondary	до 5–10 сек	Информационные данные, их задержка не критична.
Отображение статуса заказа	secondary	до 2 сек	Довольно критичная информация.
Проверка статуса перед выполнением логистики или списанием	primary		Нужны строго актуальные данные.
Создание уникальной активной корзины	primary		Чтобы не плодить дубликаты корзин, которые могут создать неконсистентность при дальнейших операциях пользователя с корзиной.
Получение активной корзины	primary		Пользователь должен видеть отражение своих действий по добавлению\изменению кол-ва\удалению товаров в корзине.
Получение не активной\архивной корзины	secondary	до 30-60 сек	Это служебная информация, допустима задержка.
Проверка корзины перед оформлением заказа.	primary		Формирование заказа должно быть на строго актуальных данных.

Итого:

- Всё, что влияет на покупку товара: остаток, деньги - читаем только с primary.
- Справочная \ историческая \ кешируемая информацию - читаем с secondary.

Название задачи:

Онлайн-магазин «Мобильный мир» - Миграция на Cassandra: модель данных, стратегии репликации и шардирования

Специфика Cassandra:

- репликация без ведущей ноды;
- есть механизмы восстановления согласованности;
- Consistent Hashing, шардирование на кольце токенов;
- AP: Availability - Partition Tolerance;
- eventual consistency;
- устойчивость к большим нагрузкам на запись;
- хорошо масштабируется горизонтально; быстрый решардинг.

Нефункциональные требования

№	Требование
1.	Высокая отказоустойчивость (leaderless-репликация).
2.	Быстрое горизонтальное масштабирование без полного перераспределения данных.
3.	Равномерное распределение данных.

Решение

Анализ требований к устойчивости

Сущность	Осн. операции	Частота обновлений	Критичность доступности (запись)	Критичность целостности	Комментарий
Заказы (актуальные)	- Быстрое создание заказов с одновременным списанием остатков. - Отображение статуса.	Средняя \ очень высокая при пиковых нагрузках	Высокая	Высокая	Требуется сильная атомарность, применение Cassandra сомнительно.

Сущность	Осн. операции	Частота обновлений	Критичность доступности (запись)	Критичность целостности	Комментарий
Товары (каталог)	- Поиск товаров по категориям и фильтрация по диапазону цен. - Описание товара на странице продукта	Низкая	Средняя	Низкая	Cassandra не даёт преимуществ для статичного каталога.
Остатки	- Частые обновления остатков при покупках.	Очень высокая	Очень высокая	Очень высокая	Требуется сильная атомарность, применение Cassandra сомнительно.
Корзины	- Создание корзины, когда заходит гость или новый пользователь. - Получение текущей корзины. - Операции с товарами. - Изменение статуса. - Удаление архивных корзин.	Высокая \ очень высокая при пиковых нагрузках	Высокая	Средняя	Перенести в Cassandra: требуется высокая доступность записи.
История заказов	- Поиск истории заказов конкретного пользователя. - Завершенные заказы присоединяются к коллекции.	Средняя	Средняя	Средняя	Перенести в Cassandra: коллекция растет монотонно, при масштабировании данные будут легче перераспределяться, чем на Mongo.
Пользовательские сессии	- Сессии, события, действия на сайте.	Очень высокая	Средняя	Средняя	Перенести в Cassandra: коллекция растет монотонно, при масштабировании данные будут легче перераспределяться, чем на Mongo.

Итого: предполагаем гибридную архитектуру: сущности, создающие нагрузку на запись \ участвующие в массовых перемещениях, вынесем в Cassandra. Сущности, чувствительные к целостности, оставим в MongoDB (с учетом рекомендаций, данных в предыдущих разделах).

Описание сущностей в Cassandra

```
CREATE TABLE carts_by_user (  
    user_id UUID,  
    cart_id UUID,  
    status TEXT,  
    items LIST<FROZEN<cart_item>>,  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP,  
    PRIMARY KEY (user_id, status, cart_id)  
) WITH CLUSTERING ORDER BY (status ASC, cart_id DESC);
```

```
CREATE TABLE carts_by_session (  
    session_id UUID,  
    cart_id UUID,  
    status TEXT,  
    items LIST<FROZEN<cart_item>>,  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP,  
    PRIMARY KEY (session_id, status, cart_id)  
) WITH CLUSTERING ORDER BY (status ASC, cart_id DESC);  
    AND default_time_to_live = 2592000; -- 30 дней
```

```
CREATE TABLE order_history (  
    user_id UUID,  
    created_at TIMESTAMP,  
    order_id UUID,  
    status TEXT,  
    total DECIMAL,  
    geozone TEXT,  
    items LIST<FROZEN<order_item>>,  
    PRIMARY KEY (user_id, created_at)  
) WITH CLUSTERING ORDER BY (created_at DESC);
```

```
CREATE TABLE user_sessions (  
    session_id UUID,  
    event_time TIMESTAMP,  
    event_type TEXT,  
    payload MAP<TEXT, TEXT>,  
    PRIMARY KEY (session_id, event_time)  
) WITH CLUSTERING ORDER BY (event_time DESC)  
    AND default_time_to_live = 2592000; -- 30 дней
```

Оценка рисков

Сущность	Таблица	Partition key	Размер партиции	Риск горячих партиций	Комментарий
Корзины	carts_by_user \ carts_by_session	user_id UUID \ session_id UUID	Одна корзина ≈ 1–10 КБ + очистка по ttl => приемлемо	нет (если не бот)	
История заказов (только завершенные)	order_history	user_id UUID	При 1000 заказов/ пользователя и 1 КБ на заказ => ~1 МБ. При 100 000 заказов - 100 МБ (рекомендуемый предел партиции)	нет (заказ может делать только авторизованный пользователь, что минимизирует риск фрода)	Можно добавить бакет PRIMARY KEY ((user_id, bucket), created_at), bucket = year
Пользовательские сессии	user_sessions	session_id UUID	При 10 000 событий/сессию и 200 байт на событие => ~2 МБ + очистка по ttl => приемлемо	нет (если не бот)	

- UUID - случайный, высококардинальный идентификатор => минимизация горячих партиций и равномерное распределение.
- consistent hashing + vnodes гарантирует, что при добавлении узла данные перераспределяются локально, без глобальной перебалансировки (в отличие от MongoDB range-based sharding).

Рекомендации по consistency level и стратегиям обеспечения целостности

Сущность	Таблица	CL (Write)	CL (Read)	Hinted Handoff	Read Repair	Anti-Entropy Repair	Обоснование
Корзины в UI	carts_by_*	ONE \ QUORUM	QUORUM	Да	Да	Да	У нас 2 основ сценария чте корзин: показ UI и при созд заказа. В слу возможно сто пренебречь р рассинхрона реплик, и использовать CL(Write)=ON (особенно при пиковых нагр Anti-Entropy F

Сущность	Таблица	CL (Write)	CL (Read)	Hinted Handoff	Read Repair	Anti- Entropy Repair	Обоснова
							предлагается выполнять из расчета, что г архивным кор еще пройдет скрипт анали
Корзины перед checkout	carts_by_user	QUORUM	QUORUM	Да	Да	Да	При чтении к перед оформление заказа - важн получить ее актуальное состояние. Д этого можно выполнить за статуса с CL(Write)=QU А также заблокироват корзину от изменений с помощью Lightweight Transaction н время создан заказа в транзакционн СУБД.
История заказов (только завершенные)	order_history	QUORUM	QUORUM	Да	Да	Да	Данные долгоживущи чтение\записи средней интенсивност требуется актуальное состояние (пользовател должен виде правильную историю свои заказов).
Пользовательские сессии	user_sessions	ONE	ONE	Да	-	Да	Очень высока частота запис чтение редко (аналитика,

Сущность	Таблица	CL (Write)	CL (Read)	Hinted Handoff	Read Repair	Anti- Entropy Repair	Обоснова
							отладка); низ критичность целостности. — максимаь пропускная способность.

--